

RepoLens: A Local-First Code Analysis and Documentation Generation System for Large Repositories

Member 1: Yuhang Zhao Student ID: 2025234377
Member 2: Xuebing Li Student ID: 2025233168
Member 3: Jinxi Xiao Student ID: 2025233210

June 16, 2026

Abstract

Understanding a large and unfamiliar code repository is a recurring challenge in software engineering. Manual documentation is often missing, outdated, or inconsistent with the actual source. While large language models (LLMs) can summarize code, they may hallucinate function signatures, parameter types, and exception specifications that do not exist in the source. This report presents RepoLens, a local-first system that combines tree-sitter-based static analysis with local LLM inference to produce traceable, verifiable API documentation and to support retrieval-augmented question answering. The system ingests a GitHub repository, builds hybrid vector-and-symbol indexes, generates a hierarchical MkDocs documentation site, and provides an interactive QA agent equipped with sandboxed file-inspection tools. All inference runs against a local llama.cpp server using a quantized GGUF coding model, eliminating cloud API dependencies. The project demonstrates that grounding LLM output in deterministically extracted source facts significantly reduces hallucination risk while preserving readable, informative documentation.

1 Introduction

Modern software repositories can span thousands of files, hundreds of modules, and millions of lines of code. When a developer joins a new project, contributes to an unfamiliar open-source library, or audits a legacy codebase, the time spent navigating source code to build a mental model of the architecture is often the dominant cost. Well-maintained API documentation can dramatically reduce this overhead, but in practice documentation frequently lags behind code changes, covers only a subset of the public API, or is missing entirely.

Large language models offer a promising avenue for automated documentation generation. Given source code as input, an LLM can produce natural-language explanations of module responsibilities, function behavior, and architectural patterns. However, using an LLM as a black-box documentation generator carries significant risks. Models may invent parameter names, return types, or raised exceptions that are plausible yet factually incorrect—the hallucination problem. In safety-critical or production settings, such fabricated details can mislead developers and erode trust in generated doc-

umentation.

RepoLens addresses this problem through a design principle of **deterministic extraction, semantic explanation**. Source-code facts—symbol names, function signatures, line ranges, call relationships, and actual **raise** statements—are extracted by tree-sitter-based static analysis and treated as authoritative. The LLM is then tasked with writing human-readable descriptions, module-level narratives, and architectural summaries, working strictly within the constraints provided by the extracted facts. This separation of concerns mirrors the Unix philosophy: each component has a focused responsibility, and the full pipeline is composed through a simple command-line interface.

The system runs entirely on local hardware using a quantized GGUF model served by llama.cpp with Vulkan backend acceleration. This design eliminates reliance on cloud API services, which can introduce latency, cost, privacy concerns, and network-dependency issues. The project has been tested on moderate-to-large Python repositories and includes initial experiments with C++ codebases.

2 System Overview and Design Goals

RepoLens was designed with the following concrete goals:

1. **Automated ingestion:** Accept a GitHub URL or local directory, clone or locate the repository, and produce a structured representation of all source files and their symbols.
2. **Dual indexing:** Build both a semantic vector index (for natural-language queries) and a precise symbol index (for exact name lookups), stored persistently for reuse across sessions.
3. **Multi-level documentation:** Generate three tiers of documentation—Architecture overview, Module guides, and Detailed API references—rendered as a browsable MkDocs Material site.
4. **Interactive QA:** Provide a retrieval-augmented agent that can inspect source files through sandboxed tools and answer questions with `file:line` citations.
5. **Local-first inference:** Use a local llama.cpp server with a quantized model, supporting both managed (auto lifecycle) and external-server modes.

6. **Resumable generation:** Support incremental caching and safe-restart patterns so long-running documentation jobs can survive memory pressure and process interruptions.

3 Architecture and Implementation

The high-level pipeline proceeds as follows:

```
GitHub / local repository
-> ingestion (clone, scan, parse)
-> indexing (chunk, embed, store)
-> docgen (lens, summarize, render)
-> MkDocs documentation site
-> QA agent / Chainlit web UI
```

The unified command-line entry point is implemented in `cli.py` using the Typer framework. The five principal commands are `index`, `lens`, `doc`, `ask`, and `skill`. The following subsections describe each stage of the pipeline in detail.

3.1 Repository Ingestion and Static Analysis

The `ingestion` package handles the initial processing of a target repository. The module `ingestion.clone` accepts either a GitHub URL or a local directory path and resolves it to a local repository root, using shallow cloning (`depth=1`) for remote sources to minimize disk usage. The module `ingestion.filetree` walks the repository tree and identifies source files by extension, filtering out common non-code directories (e.g., `.git`, `node_modules`) and binary files.

The core parsing logic resides in `ingestion.parse`. It uses the `tree-sitter-language-pack` to support multiple programming languages through a unified API. For each source file, the appropriate tree-sitter language grammar is loaded and the AST is traversed with a recursive visitor. The system extracts the following deterministic facts for each symbol:

- **Name and kind:** The symbol’s declared name and its category (function, class, method, etc.).
- **Signature:** The exact source text from the declaration start to the body start, captured verbatim.
- **Source location:** File path, start line, and end line.
- **Docstring:** Any associated documentation comment, when available.
- **Calls:** Function names referenced within the symbol’s body, used for call-graph construction.
- **Raised exceptions:** Explicit `raise` or `throw` statements found within the body.

A notable design choice is that the implementation avoids tree-sitter `.scm` query files in favor of a straightforward per-language mapping from AST node types to symbol kinds, combined with a generic recursive visitor. This keeps the parsing layer simple and easy to extend to new languages. Single-file parse errors are silently skipped (graceful degradation), and the output is a `RepoMap` dataclass containing a flat list of `FileInfo` objects, each holding its own list of `SymbolInfo` records.

3.2 Index Construction

The `indexing` package transforms the parsed `RepoMap` into two complementary search indexes, orchestrated by `indexing.builder.build_index`. All index data is persisted under `.cache/index/<repo_name>/`.

3.2.1 Code-Aware Chunking

The module `indexing.chunk` divides source files into semantic chunks suitable for embedding. The chunking strategy is code-aware rather than line-count-based: top-level functions become individual chunks; classes are split into a header chunk plus independent method chunks (avoiding excessively large class blobs); and module-level code between symbols is chunked with a sliding window of 80 lines, capped at 200 lines per chunk. Each chunk’s text is decorated with a structured header containing language, kind, qualified name, and signature metadata to improve embedding quality.

3.2.2 Vector and Symbol Indexes

Chunks are embedded using `fastembed` with the BAAI/bge-small-en-v1.5 model running on CPU (reserving the GPU for LLM inference). The resulting 384-dimensional vectors are stored in a persistent ChromaDB collection with cosine distance. In parallel, all extracted symbols are inserted into a SQLite database (`symbols.db`) with columns for name, qualified name, kind, file path, module, signature, and docstring, enabling exact and prefix-based symbol lookup.

3.2.3 Hybrid Retrieval

The `indexing.retrieve.Retriever` class combines vector and symbol search results through rule-based re-ranking. Symbol hits receive a 0.20 relevance bonus; function, method, and class kinds receive 0.08; test files incur a 0.35 penalty. Query expansion adds CamelCase-split tokens and domain-specific synonyms (e.g., “parse argument” expands to “parse_args”, “OptionParser”). The combined approach significantly improves retrieval precision compared to vector-only or symbol-only baselines.

3.3 Documentation Generation

The `docgen` package produces a three-tier documentation hierarchy, rendered as an MkDocs Material site under `docs/<repo_name>/`.

3.3.1 Module Lens

Before full documentation generation, `docgen.lens` generates a lightweight analysis perspective for each top-level module. A lens is a JSON object specifying the module’s role, key focus points, and suggested documentation sections. Lenses are written to `lenses.json` for optional human review and editing, and marked with a source tag (`ai`, `human`, or `fallback`). During documentation generation, only AI- and human-sourced lenses

are injected into the per-module prompts. This mechanism reduces hallucinated module boundaries and improves coverage in complex repositories.

3.3.2 Map-Reduce Summarization

Module-level and architecture-level documentation is generated through a map-reduce strategy in `docgen.summarize`. In the map phase, each module is processed independently with a prompt that includes real source snippets, symbol lists, and (optionally) the module’s lens. The reduce phase concatenates all module summaries and prompts the LLM to synthesize a repository-level Architecture overview covering project structure, data flow, call chains, a module index table, and dependency relationships.

3.3.3 Detailed API Documentation

The `docgen.apidoc` module generates per-symbol API entries. Each symbol receives an LLM-generated description of its parameters, return values, exceptions, and behavior. Crucially, the function signature is *never* generated by the LLM—it is rendered directly from the statically extracted source text. Similarly, the list of raised exceptions is constrained to those actually found in the symbol’s body during parsing. This design eliminates the two most common sources of hallucination in code documentation tools.

API generation is batched with incremental persistence every 15 symbols, and each symbol’s result is cached by an MD5 fingerprint of its kind and signature. Re-running the documentation command regenerates entries only for symbols whose signatures have changed. The system also supports a “safe loop” pattern: when memory approaches a configurable threshold, current progress is saved to disk, the llama-server process is terminated to release GPU memory, the server is restarted, and generation resumes from where it left off.

3.3.4 MkDocs Rendering

The `docgen.render` module assembles all generated content into a complete MkDocs Material site, producing architecture, module, and API pages, plus read-only source pages used as link targets for API “source” references. The rendering layer includes extensive Markdown post-processing to improve output stability: table alignment normalization, duplicate separator removal, heading localization (mapping common English headings to Chinese when the LLM ignores the prompt’s language instruction), and detection of truncated type expressions. The site configuration supports hierarchical navigation with collapsible subfolder groups via the `--nav-subfolders` option.

3.4 Local Inference Service

The `inference` package provides a clean abstraction over the local LLM backend. The `LlamaClient` class wraps the OpenAI Python SDK, making the local llama-server appear as a standard OpenAI-compatible endpoint. Every API call passes `cache_prompt=True`, which

instructs llama.cpp to reuse KV-cache entries for the system prompt across calls—a critical optimization that significantly reduces prefill latency during batch generation.

The `LlamaServer` class manages the lifecycle of the llama.cpp process, supporting start, stop, restart, and health-check operations, and implements the context-manager protocol. The restart capability is essential for the managed documentation mode.

The reference deployment uses the Qwen3-Coder-30B-A3B-Instruct model in Q4_K_M quantization, served via llama.cpp’s Vulkan backend on AMD Ryzen AI Max+395 / Radeon 8060S hardware with 131,072 tokens of context window. The system is backend-agnostic, however, and can work with any OpenAI-compatible inference server.

3.5 Question Answering Agent

The QA system in `qa.agent.QAagent` implements a tool-calling loop. On each turn, the agent receives the conversation history plus a stable repository overview prefix (kept byte-identical across calls to leverage KV-cache reuse) and outputs a structured JSON action. The available tools are:

- `search_code(query, k)`: Hybrid retrieval over the indexed codebase.
- `find_symbol(name)`: Exact symbol lookup in the SQLite index.
- `read_file(path, start, end)`: Read source file lines within a range.
- `grep(pattern)`: Regex search over repository files.
- `list_dir(path)`: Browse the repository file tree.
- `final(answer)`: Submit the final answer with mandatory `file:line` citations.

All file-access tools are enforced by `qa.tools.RepoTools`, which resolves paths against the repository root and blocks traversal outside it. Read operations are capped (90 lines per read, 20 grep results, 120 directory entries) to keep context manageable. The agent is limited to 6 tool-calling iterations per question, with deduplication of repeated tool calls.

A key quality feature is the answer verification step: the `final` action is only accepted if the answer includes at least one `file:line` reference, ensuring that every answer is traceable back to the source code. For follow-up questions, the agent maintains memory of previous answers, extracted code items, and referenced files, enabling multi-turn conversations that build on prior context. The web interface is provided by a Chainlit application in `app.py`, which displays the tool-call trajectory alongside the final answer and can be embedded directly into the generated MkDocs site.

4 Related Work

Several categories of tools address similar problems. Traditional documentation generators such as Doxygen and Sphinx rely on code comments and annotations, producing reference-style output with minimal narrative

content. AI-assisted tools like GitHub Copilot Chat and Sourcegraph Cody can answer code questions but require cloud connectivity and do not generate persistent documentation sites. Academic systems such as RepoFusion and RepoGraph have explored repository-level code understanding with LLMs, but typically focus on code generation tasks rather than documentation. RepoLens differs in three main ways: (1) it generates a complete, browsable documentation site rather than providing only ephemeral chat responses; (2) it enforces strict separation between deterministically extracted facts and LLM-generated descriptions; and (3) it is designed to run entirely on local hardware with no external service dependencies.

5 Evaluation and Observations

The system has been tested on several repositories of varying size and complexity. The key observations are summarized below.

- **Small Python projects (e.g., Click):** The base summarization prompt produces useful documentation on its own. The lens mechanism provides only marginal improvement, as the module structure is simple and well-defined.
- **Complex Python projects (e.g., Scrapy):** The lens mechanism meaningfully improves module distinction, increases coverage of secondary modules, and reduces hallucinated content by providing the LLM with explicit per-module guidance. The map-reduce strategy handles the large symbol count effectively.
- **Large C++ projects (e.g., Ceres Solver):** C++ extraction remains the primary bottleneck. Templates, traits, macros, and complex type systems produce noisy symbol data that degrades both indexing quality and documentation accuracy. Prompt engineering alone cannot compensate for extraction noise.

The system’s memory monitoring module in `common.gpu` has proven essential for the AMD APU deployment, where video memory and system memory are separate pools. On this hardware, system memory is often the tighter constraint, and the watchdog correctly identifies and responds to both memory sources through a unified interface that queries NVIDIA SMI, AMD sysfs VRAM/GTT, and `/proc/meminfo` simultaneously.

6 Limitations and Future Work

Despite its strengths, RepoLens has several limitations that suggest directions for future improvement:

1. **C++ parsing quality:** Templates, macros, and complex inheritance hierarchies are not handled well by the current tree-sitter visitor. A dedicated C++ extraction pipeline with macro expansion and template resolution is needed for production-quality results.
2. **Scalability:** Repositories with tens of thousands of symbols require many hours of LLM inference for full API coverage. More aggressive sampling, incremental update strategies, or tiered coverage (e.g., documenting only public APIs) could reduce this cost.

3. **Evaluation methodology:** The current evaluation is qualitative and based on manual inspection. Developing automated metrics for documentation quality—such as factuality against source code, completeness of API coverage, and readability—would enable systematic comparison and regression testing.
4. **Multi-language optimization:** While the parsing layer supports multiple languages through tree-sitter, the chunking and prompting strategies have been optimized primarily for Python. Adapting these to other language ecosystems (Rust’s module system, Java’s package conventions) would broaden applicability.
5. **Incremental updates:** Currently, any source code change requires re-running the full pipeline. Supporting incremental index updates and targeted documentation regeneration would make the system more practical for actively developed repositories.

7 Conclusion

This report presented RepoLens, a local-first system for repository analysis, API documentation generation, and code question answering. The project demonstrates a practical approach to mitigating LLM hallucination in code documentation by combining deterministic static analysis with constrained natural-language generation. The system ingests repositories through tree-sitter parsing, builds hybrid indexes for efficient retrieval, generates three-tier MkDocs documentation, and provides an interactive QA agent with sandboxed source inspection tools, all running on a local llama.cpp server.

The key technical contributions include: (1) code-aware chunking that preserves semantic boundaries; (2) hybrid vector-and-symbol retrieval with rule-based re-ranking; (3) a lens mechanism for customizable per-module documentation prompts with human-in-the-loop editing; (4) a safe-loop pattern for resumable long-running generation under memory constraints; and (5) strict separation between deterministically extracted source facts and LLM-generated descriptions. The system has been successfully deployed on AMD APU hardware with a quantized 30B-parameter coding model, demonstrating that practical-quality code documentation can be generated entirely on local consumer hardware.

References

- [1] Tree-sitter, “An incremental parsing system for programming tools,” [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [2] Chroma, “The AI-native open-source embedding database,” [Online]. Available: <https://www.trychroma.com/>
- [3] MkDocs, “Project documentation with Markdown,” [Online]. Available: <https://www.mkdocs.org/>

- [4] llama.cpp, “LLM inference in C/C++,” [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [5] Chainlit, “Build conversational AI applications,” [Online]. Available: <https://docs.chainlit.io/>
- [6] Qdrant, “FastEmbed: Fast, lightweight embedding generation for Python,” [Online]. Available: <https://github.com/qdrant/fastembed>
- [7] D. Shrivastava et al., “RepoFusion: Training Code Models to Understand Your Repository,” arXiv:2306.10957, 2024.
- [8] S. Ouyang et al., “RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph,” arXiv:2410.14684, 2024.